

Problem Statement 1: Apply basic R programming concepts to develop a program that creates a numeric vector of marks for 10 students, uses conditional statements to classify each student as *Distinction*, *First Division*, *Second Division*, *Pass*, or *Fail*, and displays the results using a loop. Further, install and load the `ggplot2` package and generate a bar chart to visualize the students' marks.

Objective: The objective of this program is to apply basic R programming concepts to create and manipulate a numeric vector of student marks. It uses conditional statements and looping constructs to classify students based on their performance and displays the results. Additionally, the program visualizes the marks of students using a bar chart with the `ggplot2` package.

R Program:

```
marks <- c(85, 72, 64, 58, 91, 47, 39, 76, 69, 82)
```

```
students <- paste("Student", 1:10)
```

```
for (i in 1:length(marks)) {  
  if (marks[i] >= 75) {  
    result <- "Distinction."  
  } else if (marks[i] >= 60) {  
    result <- "First Division."  
  } else if (marks[i] >= 50) {  
    result <- "Second Division."  
  } else if (marks[i] >= 40) {  
    result <- "Pass."  
  } else {  
    result <- "Fail."  
  }  
  cat(students[i], "- Marks:", marks[i], "- Result:", result, "\n")  
}
```

```
install.packages("ggplot2")
```

```
library(ggplot2)
```

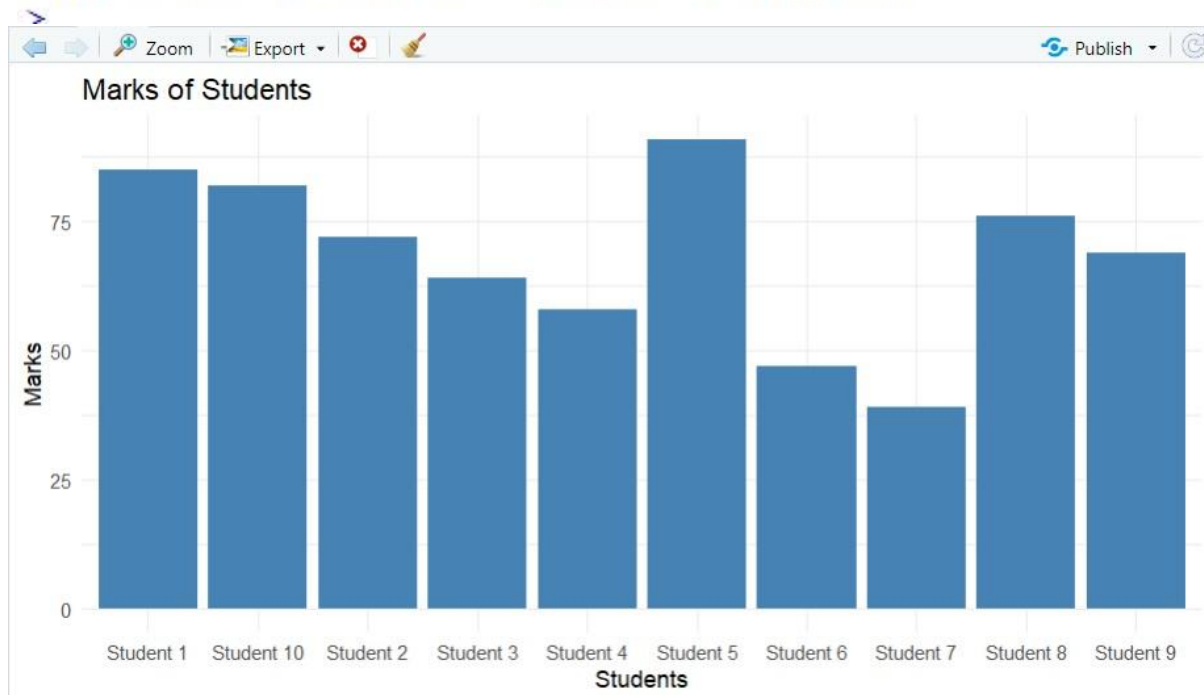
```
student_data <- data.frame(  
  Student = students,  
  Marks = marks  
)
```

```
ggplot(student_data, aes(x = Student, y = Marks)) +  
  geom_bar(stat = "identity", fill = "steelblue") +
```

```
labs(  
  title = "Marks of Students",  
  x = "Students",  
  y = "Marks"  
) +  
theme_minimal()
```

OUTPUT

```
Student 1 - Marks: 85 - Result: Distinction  
Student 2 - Marks: 72 - Result: First Division  
Student 3 - Marks: 64 - Result: First Division  
Student 4 - Marks: 58 - Result: Second Division  
Student 5 - Marks: 91 - Result: Distinction  
Student 6 - Marks: 47 - Result: Pass  
Student 7 - Marks: 39 - Result: Fail  
Student 8 - Marks: 76 - Result: Distinction  
Student 9 - Marks: 69 - Result: First Division  
Student 10 - Marks: 82 - Result: Distinction
```



Problem Statement 2: Apply appropriate R functions to compute the mean, mode, variance, and standard deviation of variables in the *mtcars* dataset and display the results.

Objective: The objective of this program is to apply built-in R functions to compute descriptive statistics such as mean, mode, variance, and standard deviation for selected variables in the *mtcars* dataset and display the results in a readable format.

R Program:

```
data(mtcars)
```

```
head(mtcars)
```

```
mpg_data <- mtcars$mpg
```

```
mean_mpg <- mean(mpg_data)
```

```
variance_mpg <- var(mpg_data)
```

```
sd_mpg <- sd(mpg_data)
```

```
get_mode <- function(x) {  
  unique_values <- unique(x)  
  unique_values[which.max(tabulate(match(x, unique_values)))]  
}
```

```
mode_mpg <- get_mode(mpg_data)
```

```
cat("Mean of mpg:", mean_mpg, "\n")
```

```
cat("Mode of mpg:", mode_mpg, "\n")
```

```
cat("Variance of mpg:", variance_mpg, "\n")
```

```
cat("Standard Deviation of mpg:", sd_mpg, "\n")
```

OUTPUT

```
> cat("Mean of mpg:", mean_mpg, "\n")
Mean of mpg: 20.09062
> cat("Mode of mpg:", mode_mpg, "\n")
Mode of mpg: 21
> cat("Variance of mpg:", variance_mpg, "\n")
Variance of mpg: 36.3241
> cat("Standard Deviation of mpg:", sd_mpg, "\n")
Standard Deviation of mpg: 6.026948
\ |
```

Problem Statement 3: Load a dataset from a CSV file in R and generate a bar chart, pie chart, and 3D pie chart to visualize the data.

Objective: The objective of this program is to load data from a CSV file into R and visualize the data using different graphical techniques, such as bar charts, pie charts, and a 3D pie chart for better understanding and comparison.

R Program:

```
setwd("C:/Users/public/Documents")
```

```
data <- read.csv("sales.csv")
```

```
print(data)
```

```
categories <- data[, 1]
```

```
values <- data[, 2]
```

```
barplot(  
  values,  
  names.arg = categories,  
  col = "skyblue",  
  main = "Bar Chart Representation",  
  xlab = "Categories",  
  ylab = "Values"  
)
```

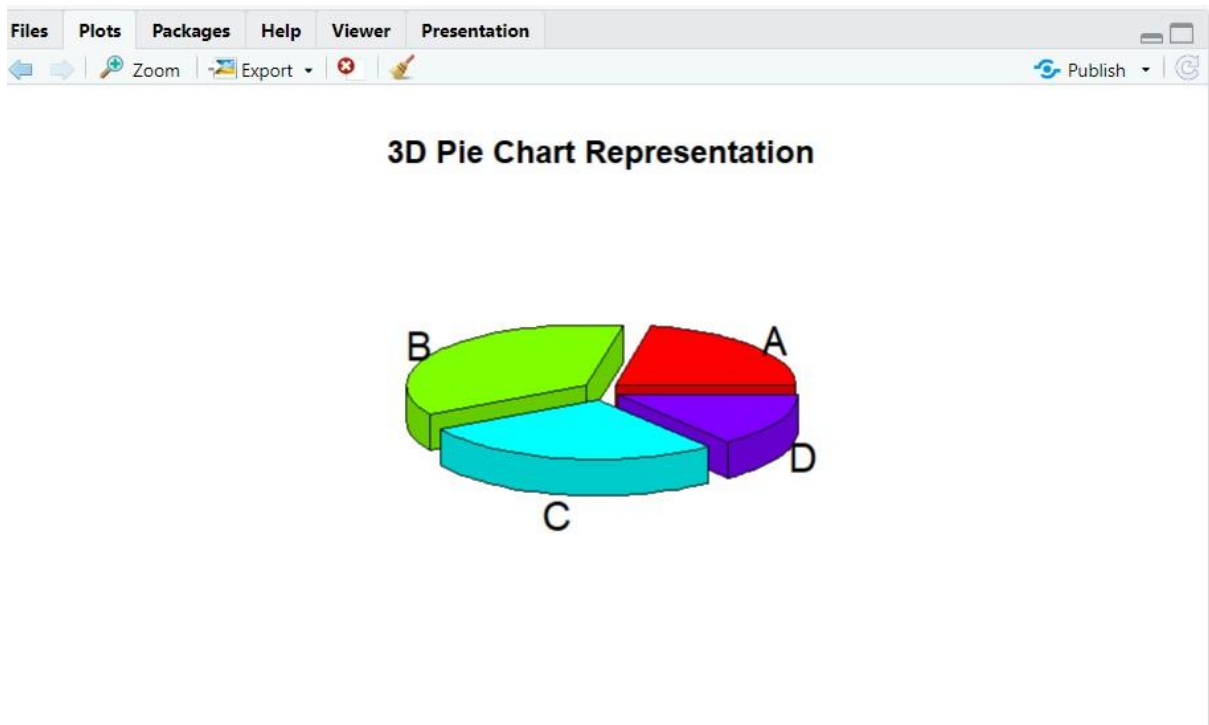
```
pie(  
  values,  
  labels = categories,  
  col = rainbow(length(values)),  
  main = "Pie Chart Representation"  
)
```

```
install.packages("plotrix")
```

```
library(plotrix)
```

```
pie3D(  
  values,  
  labels = categories,  
  explode = 0.1,  
  main = "3D Pie Chart Representation"  
)
```

OUTPUT



Problem Statement 4: Analyze a given CSV dataset to identify missing and inconsistent values and perform data cleaning using suitable R functions.

Objective: The objective of this program is to analyze a CSV dataset to identify missing and inconsistent values and perform data cleaning using appropriate R functions to improve data quality for further analysis.

R Program:

```
setwd("C:/Users/public/Documents")
```

```
data <- read.csv("employee.csv", stringsAsFactors = FALSE)
```

```
str(data)
```

```
summary(data)
```

```
colSums(is.na(data))
```

```
data[!complete.cases(data), ]
```

```
data$Salary[is.na(data$Salary)] <- mean(data$Salary, na.rm = TRUE)
```

```
data$Department[data$Department == "" | is.na(data$Department)] <- "Unknown"
```

```
data$Age[data$Age <= 0] <- NA
```

```
data$Age[is.na(data$Age)] <- mean(data$Age, na.rm = TRUE)
```

```
data <- unique(data)
```

```
print(data)
```

```
summary(data)
```

OUTPUT

```
<
> # Step 12: Display the cleaned dataset
> print(data)
  ID  Name      Age Department  Salary
1  1  Ravi  25.00000         HR   30000
2  2  Anita 26.66667         IT   45000
3  3   John 26.66667    Finance  36250
4  4  Rahul 30.00000    Unknown  40000
5  5   Ravi 25.00000         HR   30000
>
> # Step 13: Display summary after data cleaning
> summary(data)
      ID      Name      Age      Department      Salary
Min.   :1  Length:5  Min.   :25.00  Length:5  Min.   :30000
1st Qu.:2  Class  :character 1st Qu.:25.00  Class  :character 1st Qu.:30000
Median :3  Mode   :character Median :26.67  Mode   :character Median :36250
Mean    :3                      Mean    :26.67                      Mean    :36250
3rd Qu.:4                      3rd Qu.:26.67                      3rd Qu.:40000
Max.    :5                      Max.    :30.00                      Max.    :45000
> |
```


Problem Statement 5: Plot the given CSV dataset in both 2D and 3D formats using appropriate R plotting functions and label the axes clearly.

Objective: The objective of this program is to load a dataset from a CSV file and visualize the data using 2D and 3D plotting techniques in R, with clearly labeled axes for better understanding and interpretation.

R Program:

```
setwd("C:/Users/public/Documents")
```

```
data <- read.csv("data.csv")
```

```
print(data)
```

```
x <- data$X
```

```
y <- data$Y
```

```
z <- data$Z
```

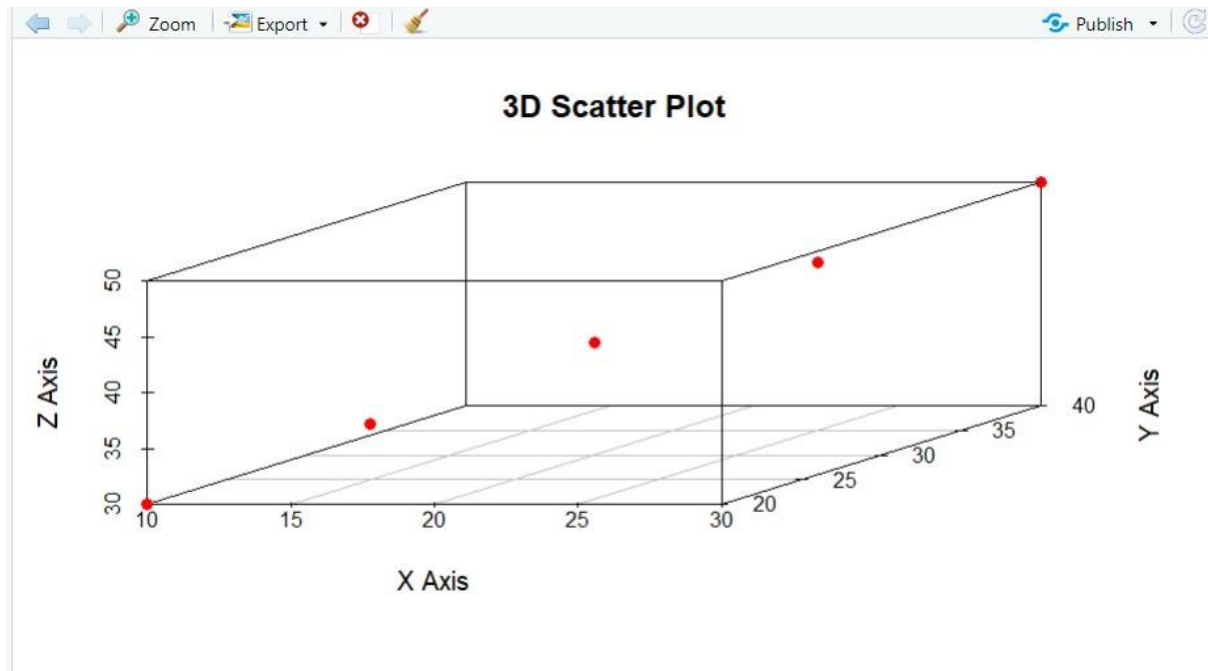
```
plot(  
  x, y,  
  main = "2D Scatter Plot",  
  xlab = "X Axis Values",  
  ylab = "Y Axis Values",  
  col = "blue",  
  pch = 19  
)
```

```
install.packages("scatterplot3d")
```

```
library(scatterplot3d)
```

```
scatterplot3d(  
  x, y, z,  
  main = "3D Scatter Plot",  
  xlab = "X Axis",  
  ylab = "Y Axis",  
  zlab = "Z Axis",  
  color = "red",  
  pch = 16  
)
```

Output



Problem Statement 6: Perform a linear regression on the following mice dataset:

Height: 140, 142, 150, 147, 139, 152, 154, 135, 148, 147

Weight: 59, 61, 66, 62, 57, 68, 69, 58, 63, 62

Derive the relationship coefficients and summarize the regression results

Objective: The objective of this program is to perform linear regression on the given mice dataset to determine the relationship between height and weight, derive the regression coefficients, and summarize the regression results using appropriate R functions.

R Program:

```
height <- c(140, 142, 150, 147, 139, 152, 154, 135, 148, 147)
```

```
weight <- c(59, 61, 66, 62, 57, 68, 69, 58, 63, 62)
```

```
mice_data <- data.frame(  
  Height = height,  
  Weight = weight  
)
```

```
print(mice_data)
```

```
reg_model <- lm(Weight ~ Height, data = mice_data)
```

```
coefficients(reg_model)
```

```
summary(reg_model)
```

Output

```
> # Step 4: Display the dataset
> print(mice_data)
  Height Weight
1    140     59
2    142     61
3    150     66
4    147     62
5    139     57
6    152     68
7    154     69
8    135     58
9    148     63
10   147     62
>
> # Step 5: Perform linear regression
> # weight is the dependent variable and Height is the independent variable
> reg_model <- lm(Weight ~ Height, data = mice_data)
>
> # Step 6: Display the regression coefficients
> coefficients(reg_model)
(Intercept)      Height
-28.9089307    0.6286722
>
>
> # Step 7: Display the summary of the regression model
> summary(reg_model)
```

Call:

```
lm(formula = weight ~ Height, data = mice_data)
```

Residuals:

Min	1Q	Median	3Q	Max
-1.5059	-1.3910	0.2515	0.9794	2.0382

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	-28.90893	11.14159	-2.595	0.0319 *
Height	0.62867	0.07657	8.211	3.62e-05 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1.413 on 8 degrees of freedom

Multiple R-squared: 0.8939, Adjusted R-squared: 0.8807

F-statistic: 67.42 on 1 and 8 DF, p-value: 3.619e-05

Problem Statement 7: Apply logistic regression to the Iris dataset, creating a subset with only two species, and interpret the results.

Objective: The objective of this program is to apply logistic regression on the Iris dataset by creating a subset containing only two species and to analyze and interpret the relationship between flower measurements and species classification.

R Program:

```
data(iris)
```

```
str(iris)
```

```
iris_subset <- subset(iris, Species != "virginica")
```

```
iris_subset$Species <- factor(iris_subset$Species)
```

```
head(iris_subset)
```

```
logistic_model <- glm(  
  Species ~ Sepal.Length + Sepal.Width,  
  data = iris_subset  
)
```

```
summary(logistic_model)
```

```
predicted_probabilities <- predict(logistic_model, type = "response")
```

```
predicted_class <- ifelse(predicted_probabilities > 0.5, "versicolor", "setosa")
```

```
table(Predicted = predicted_class, Actual = iris_subset$Species)
```

Output

```
Call:
glm(formula = Species ~ Sepal.Length + Sepal.Width, family = binomial,
     data = iris_subset)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-360.6	195972.9	-0.002	0.999
Sepal.Length	131.8	64577.0	0.002	0.998
Sepal.Width	-110.1	55361.5	-0.002	0.998

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1.3863e+02 on 99 degrees of freedom
Residual deviance: 7.1185e-09 on 97 degrees of freedom
AIC: 6

Number of Fisher Scoring iterations: 25

```
>
> # Step 8: Predict probabilities using the model
> predicted_probabilities <- predict(logistic_model, type = "response")
>
> # Step 9: Convert probabilities into predicted classes
> predicted_class <- ifelse(predicted_probabilities > 0.5,
+                           "versicolor", "setosa")
>
> # Step 10: Create a confusion matrix
> table(Predicted = predicted_class, Actual = iris_subset$Species)
```

	Actual	
Predicted	setosa	versicolor
setosa	50	0
versicolor	0	50

Problem Statement 8: Implement and apply the KNN classification algorithm on a large dataset using R.

Objective: The objective of this program is to implement and apply the K-Nearest Neighbors (KNN) classification algorithm on a dataset using R, classify observations based on similarity, and evaluate the classification results.

R Program:

```
library(class)
```

```
data(iris)
```

```
str(iris)
```

```
features <- iris[, 1:4]
```

```
class_labels <- iris$Species
```

```
normalize <- function(x) {  
  return((x - min(x)) / (max(x) - min(x)))  
}
```

```
features_normalized <- as.data.frame(lapply(features, normalize))
```

```
set.seed(123)
```

```
train_index <- sample(1:nrow(features_normalized), 0.7 * nrow(features_normalized))
```

```
train_data <- features_normalized[train_index, ]
```

```
test_data <- features_normalized[-train_index, ]
```

```
train_labels <- class_labels[train_index]
```

```
test_labels <- class_labels[-train_index]
```

```
knn_predictions <- knn(  
  train = train_data,  
  test = test_data,  
  cl = train_labels,  
  k = 5  
)  
  
print(knn_predictions)  
  
confusion_matrix <- table(Predicted = knn_predictions, Actual = test_labels)  
print(confusion_matrix)  
  
accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)  
  
cat("Classification Accuracy:", accuracy * 100, "%\n")
```

Output

```
>  
> # Step 8: Display the predicted classes  
> print(knn_predictions)  
[1] setosa      setosa      setosa      setosa      setosa      setosa      setosa  
[8] setosa      setosa      setosa      setosa      setosa      setosa      setosa  
[15] versicolor versicolor versicolor versicolor versicolor versicolor versicolor  
[22] versicolor versicolor versicolor versicolor versicolor versicolor virginica  
[29] versicolor versicolor versicolor versicolor virginica virginica virginica  
[36] virginica  virginica  virginica  virginica  virginica  virginica  versicolor  
[43] virginica  virginica  virginica  
Levels: setosa versicolor virginica  
>  
> # Step 9: Create a confusion matrix  
> confusion_matrix <- table(Predicted = knn_predictions, Actual = test_labels)  
> print(confusion_matrix)  
      Actual  
Predicted setosa versicolor virginica  
setosa      14           0           0  
versicolor   0          17           1  
virginica    0           1          12  
>  
> # Step 10: Calculate classification accuracy  
> accuracy <- sum(diag(confusion_matrix)) / sum(confusion_matrix)  
> cat("Classification Accuracy:", accuracy * 100, "%\n")  
Classification Accuracy: 95.55556 %
```


Problem Statement 9: Implement the ID3 algorithm to build a decision tree model using R.

Objective: The objective of this program is to implement the ID3 algorithm to build a decision tree model on a dataset, classify observations, and analyze the tree structure and accuracy.

R Program:

```
install.packages("C50")
```

```
library(C50)
```

```
data(iris)
```

```
iris$Species <- as.factor(iris$Species)
```

```
str(iris)
```

```
set.seed(123)
```

```
train_index <- sample(1:nrow(iris), 0.7 * nrow(iris))
```

```
train_data <- iris[train_index, ]
```

```
test_data <- iris[-train_index, ]
```

```
tree_model <- C5.0(Species ~ ., data = train_data)
```

```
summary(tree_model)
```

```
predictions <- predict(tree_model, test_data)
```

```
conf_matrix <- table(Predicted = predictions, Actual = test_data$Species)
```

```
print(conf_matrix)
```

```
accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)
```

```
cat("Decision Tree Accuracy:", accuracy * 100, "%\n")
```

Output

Call:

```
C5.0.formula(formula = Species ~ ., data = train_data)
```

C5.0 [Release 2.07 GPL Edition]

Tue Feb 3 11:32:54 2026

Class specified by attribute 'outcome'

Read 105 cases (5 attributes) from undefined.data

Decision tree:

```
Petal.Length <= 1.9: setosa (36)
Petal.Length > 1.9:
: ...Petal.Width > 1.7: virginica (34/1)
  Petal.Width <= 1.7:
  : ...Petal.Length <= 4.9: versicolor (31/1)
    Petal.Length > 4.9: virginica (4/1)
```

Evaluation on training data (105 cases):

Decision Tree			

Size	Errors		
4	3(2.9%)	<<	
(a)	(b)	(c)	<-classified as
----	----	----	
36			(a): class setosa
	30	2	(b): class versicolor
	1	36	(c): class virginica

Attribute usage:

```
100.00% Petal.Length
65.71% Petal.Width
```

Time: 0.0 secs

```
>
> # Step 8: Make predictions on the test set
> predictions <- predict(tree_model, test_data)
>
> # Step 9: Create a confusion matrix
> conf_matrix <- table(Predicted = predictions, Actual = test_data$Species)
> print(conf_matrix)
      Actual
Predicted setosa versicolor virginica
setosa    14         0         0
versicolor 0         17         0
virginica  0         1        13
>
> # Step 10: Calculate accuracy
> accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)
> cat("Decision Tree Accuracy:", accuracy * 100, "%\n")
Decision Tree Accuracy: 97.77778 %
```

Problem Statement 10: Apply the K-means clustering algorithm to the given dataset and interpret the resulting clusters.

Objective: The objective of this program is to apply the K-Means clustering algorithm to a dataset, identify natural groupings of observations, and interpret the resulting clusters for meaningful patterns.

R Program:

```
data(iris)

iris_data <- iris[, 1:4]

iris_scaled <- scale(iris_data)

k <- 3

set.seed(123)

kmeans_result <- kmeans(iris_scaled, centers = k, nstart = 25)

print(kmeans_result$centers)

print(kmeans_result$cluster)

table(Cluster = kmeans_result$cluster, Species = iris$Species)

library(ggplot2)

iris_plot <- data.frame(iris_scaled, Cluster = factor(kmeans_result$cluster))

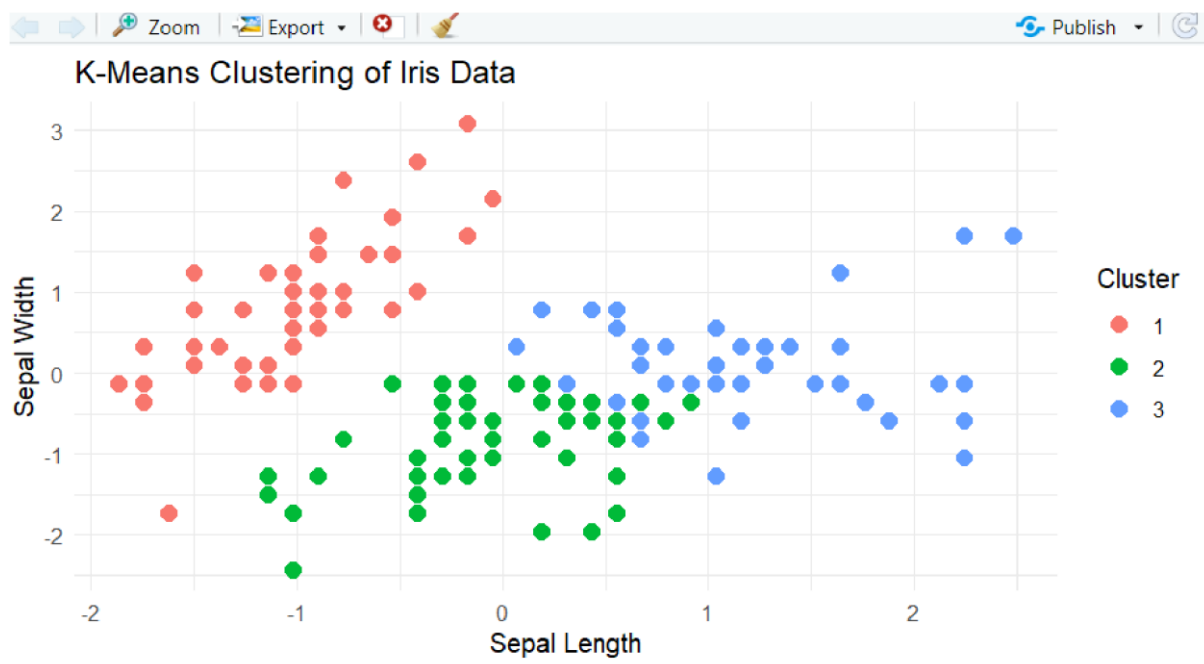
ggplot(iris_plot, aes(x = Sepal.Length, y = Sepal.Width, color = Cluster)) +
  geom_point(size = 3) +
  labs(
    title = "K-Means Clustering of Iris Data",
    x = "Sepal Length",
    y = "Sepal Width"
  ) +
  theme_minimal()
```

Output

```
>
> # Step 8: Make predictions on the test set
> predictions <- predict(tree_model, test_data)
>
> # Step 9: Create a confusion matrix
> conf_matrix <- table(Predicted = predictions, Actual = test_data$Species)
> print(conf_matrix)
```

	Actual		
Predicted	setosa	versicolor	virginica
setosa	14	0	0
versicolor	0	17	0
virginica	0	1	13

```
>
> # Step 10: Calculate accuracy
> accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)
> cat("Decision Tree Accuracy:", accuracy * 100, "%\n")
Decision Tree Accuracy: 97.77778 %
```



Problem Statement 11: Implement the CURE (Clustering Using Representatives) algorithm in R to perform clustering on a given dataset. Instead of using built-in functions, manually apply the CURE operations to partition the data into clusters.

Objective: The objective of this program is to implement the CURE (Clustering Using Representatives) algorithm manually in R to perform clustering, identify representative points for each cluster, and partition a dataset into meaningful clusters.

R Program:

```
library(stats)
```

```
data_points <- data.frame(  
  x = c(1, 2, 3, 8, 9, 10, 20, 21, 22),  
  y = c(2, 1, 3, 8, 9, 7, 20, 21, 19)  
)
```

```
k <- 3
```

```
clusters <- lapply(1:nrow(data_points), function(i) data_points[i, , drop = FALSE])
```

```
cluster_distance <- function(cluster1, cluster2) {  
  min(dist(rbind(cluster1, cluster2)))[1]  
}
```

```
num_representatives <- 2  
shrink_factor <- 0.5
```

```
while (length(clusters) > k) {
```

```
  min_dist <- Inf  
  merge_i <- merge_j <- 0
```

```
  for (i in 1:(length(clusters) - 1)) {  
    for (j in (i + 1):length(clusters)) {  
      dist_ij <- cluster_distance(clusters[[i]], clusters[[j]])  
      if (dist_ij < min_dist) {  
        min_dist <- dist_ij  
        merge_i <- i  
        merge_j <- j  
      }  
    }  
  }  
}
```

```
new_cluster <- rbind(clusters[[merge_i]], clusters[[merge_j]])
```

```
centroid <- colMeans(new_cluster)

distances_to_centroid <- apply(new_cluster, 1, function(row)
  sqrt(sum((row - centroid)^2))
)

rep_indices <- order(distances_to_centroid, decreasing = TRUE)[1:num_representatives]

representatives <- new_cluster[rep_indices, ]

representatives <- centroid + shrink_factor * (representatives - centroid)

clusters <- clusters[-c(merge_i, merge_j)]
clusters[[length(clusters) + 1]] <- representatives
}

for (i in 1:length(clusters)) {
  cat("Cluster", i, "representatives:\n")
  print(clusters[[i]])
  cat("\n")
}

plot(data_points$x, data_points$y, col = "gray", pch = 16, main = "CURE Clustering")

colors <- c("red", "blue", "green")

for (i in 1:length(clusters)) {
  points(clusters[[i]][, 1], clusters[[i]][, 2], col = colors[i], pch = 19, cex = 2)
}

legend("topleft", legend = paste("Cluster", 1:k), col = colors, pch = 19)
```

Output

Cluster 1 representatives:

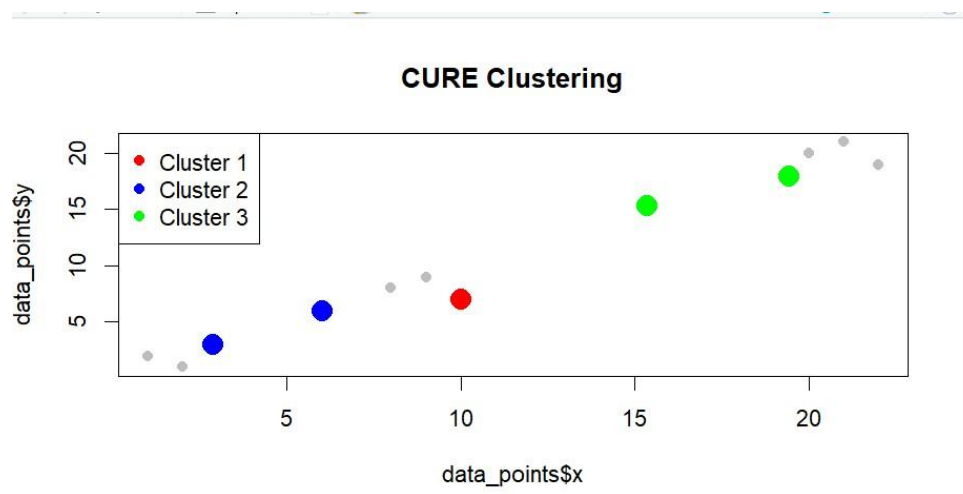
	x	y
6	10	7

Cluster 2 representatives:

	x	y
4	6.020833	6.020833
1	2.875000	3.000000

Cluster 3 representatives:

	x	y
5	15.34028	15.34028
9	19.42361	17.92361



Problem Statement 12: Implement suitable anomaly detection techniques in R for the given dataset and evaluate the detected outliers.

Objective: The objective of this program is to apply suitable anomaly detection techniques to identify outliers in a dataset and evaluate them using statistical and visualization methods.

R Program:

```
install.packages("dbscan")
```

```
library(dbscan)
```

```
data <- data.frame(  
  ID = 1:10,  
  Score = c(50, 52, 48, 55, 53, 500, 51, 49, 54, 47)  
)
```

```
print("Dataset:")  
print(data)
```

```
z_scores <- scale(data$Score)  
outliers_z <- which(abs(z_scores) > 3)
```

```
cat("Outliers detected using Z-Score:", data$ID[outliers_z], "\n")
```

```
Q1 <- quantile(data$Score, 0.25)  
Q3 <- quantile(data$Score, 0.75)  
IQR_val <- Q3 - Q1
```

```
outliers_iqr <- which(  
  data$Score < (Q1 - 1.5 * IQR_val) |  
  data$Score > (Q3 + 1.5 * IQR_val)  
)
```

```
cat("Outliers detected using IQR:", data$ID[outliers_iqr], "\n")
```

```
center <- mean(data$Score)  
cov_matrix <- var(data$Score)  
mahal_dist <- (data$Score - center)^2 / cov_matrix  
threshold <- qchisq(0.99, df = 1)
```

```
outliers_mahal <- which(mahal_dist > threshold)
```

```
cat("Outliers detected using Mahalanobis Distance:", data$ID[outliers_mahal], "\n")
```



```
lof_scores <- lof(data.frame(data$Score), k = 3)

outliers_lof <- which(lof_scores > 1.5)

cat("Outliers detected using LOF:", data$ID[outliers_lof], "\n")

boxplot(
  data$Score,
  main = "Boxplot for Outlier Detection",
  col = "lightblue",
  horizontal = TRUE
)

plot(
  data$ID, data$Score,
  pch = 19,
  col = "blue",
  main = "Outlier Detection (Z-Score)"
)

points(
  data$ID[outliers_z],
  data$Score[outliers_z],
  col = "red",
  pch = 19,
  cex = 1.5
)

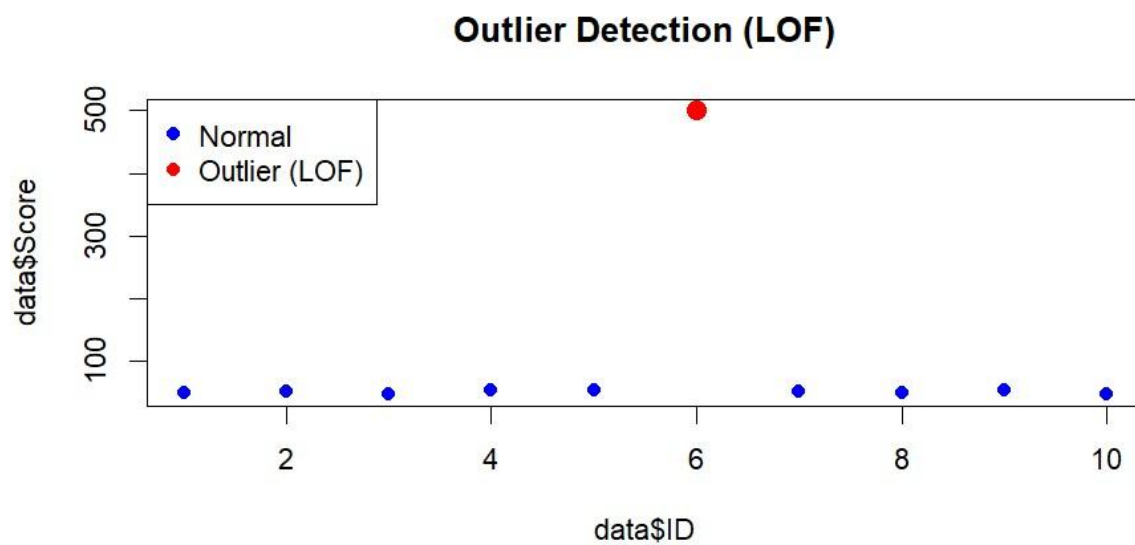
legend(
  "topleft",
  legend = c("Normal", "Outlier (Z-Score)"),
  col = c("blue", "red"),
  pch = 19
)

plot(
  data$ID, data$Score,
  pch = 19,
  col = "blue",
  main = "Outlier Detection (LOF)"
)

points(
  data$ID[outliers_lof],
```

```
data$Score[outliers_lof],  
col = "red",  
pch = 19,  
cex = 1.5  
)  
  
legend(  
  "topleft",  
  legend = c("Normal", "Outlier (LOF)"),  
  col = c("blue", "red"),  
  pch = 19  
)
```

Output



Problem Statement 13: Implement the Apriori algorithm in R to discover frequent itemsets and generate association rules from the given dataset.

Objective: The objective of this program is to apply the Apriori algorithm to discover frequent itemsets and generate association rules from transactional data using R.

R Program:

```
install.packages("arules")
```

```
library(arules)
```

```
transactions_list <- list(
```

```
  c("Milk", "Bread", "Butter"),
```

```
  c("Milk", "Bread"),
```

```
  c("Milk", "Diaper", "Beer", "Eggs"),
```

```
  c("Bread", "Butter", "Diaper"),
```

```
  c("Milk", "Bread", "Diaper", "Butter"),
```

```
  c("Bread", "Butter"),
```

```
  c("Milk", "Bread", "Butter", "Diaper")
```

```
)
```

```
trans <- as(transactions_list, "transactions")
```

```
inspect(trans)
```

```
frequent_itemsets <- apriori(trans, parameter = list(supp = 0.4, target = "frequent itemsets"))
```

```
inspect(frequent_itemsets)
```

```
rules <- apriori(trans, parameter = list(supp = 0.4, conf = 0.7, target = "rules"))
```

```
inspect(rules)
```

```
rules_sorted <- sort(rules, by = "lift", decreasing = TRUE)
```

```
inspect(rules_sorted)
```

```
install.packages("arulesViz")
```

```
library(arulesViz)
```

```
plot(rules_sorted, method = "graph", engine = "htmlwidget")
```

OUTPUT

Select by id

